Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №5

По дисциплине: «Интеллектуальный анализ данных»

Тема: «Деревья решений»

Выполнил:
Студент 4 курса
Группы ИИ-26
Яшков П. А.

Проверил:
Никитин С. К.

Брест 2026

Цель работы:

Сравнить одиночное дерево решений, случайный лес и ансамблевые алгоритмы бустинга на задаче банковского маркетинга.

Общее задание

1. Загрузить датасет Bank Marketing по варианту 11 и обработать категориальные признаки.
2. Разделить данные на обучающую и тестовую выборки.
3. Обучить Decision Tree, Random Forest, AdaBoost, CatBoost, XGBoost.
4. Оценить accuracy, precision, recall и F1 для класса подписки на депозит.
5. Сравнить модели и обосновать выбор лучшего алгоритма.

Сведения об эксперименте

Автор: Яшков П. А., группа ИИ-26, 4 курс. Вариант: 11. Все расчёты воспроизводимы: random_state/seed = 42. Исходники размещены в папке src/, результаты — src/results/.

Данные и подготовка

Использован предоставленный в папке курса набор Bank Marketing — bank.csv: 11162 записей, 16 входных полей и целевая переменная deposit («yes»/«no»). Исходные текстовые признаки (например, job, marital, education, month) преобразованы OneHotEncoder; пропуски при наличии восполняются. Признаки обучаются только на train. Тестовая выборка 25% со стратификацией по метке, random_state=42.

Важное замечание: duration — продолжительность последнего контакта — доступна только после звонка. Поэтому результаты относятся к ретроспективному прогнозу по полным данным, а не к оценке клиентов до звонка. Это ограничение учитывается при оценке практической применимости модели.

Реализация и оценка

Обучены DecisionTreeClassifier, RandomForestClassifier, AdaBoostClassifier, CatBoostClassifier, XGBClassifier. Проверка — на отложенной тестовой выборке. Для варианта 11 ключевая метрика: F1 для положительного класса deposit=yes, как компромисс precision и recall в условиях несбалансированности.

Обучение	Тест	После кодирования
8371	2791	51

Алгоритм	Accuracy	Precision yes	Recall yes	F1 yes
CatBoost	0.8628	0.8366	0.8828	0.8590
XGBoost	0.8624	0.8364	0.8820	0.8586
Random Forest	0.8570	0.8212	0.8926	0.8554
AdaBoost	0.8201	0.8144	0.8033	0.8088
Decision Tree	0.8151	0.7990	0.8147	0.8067

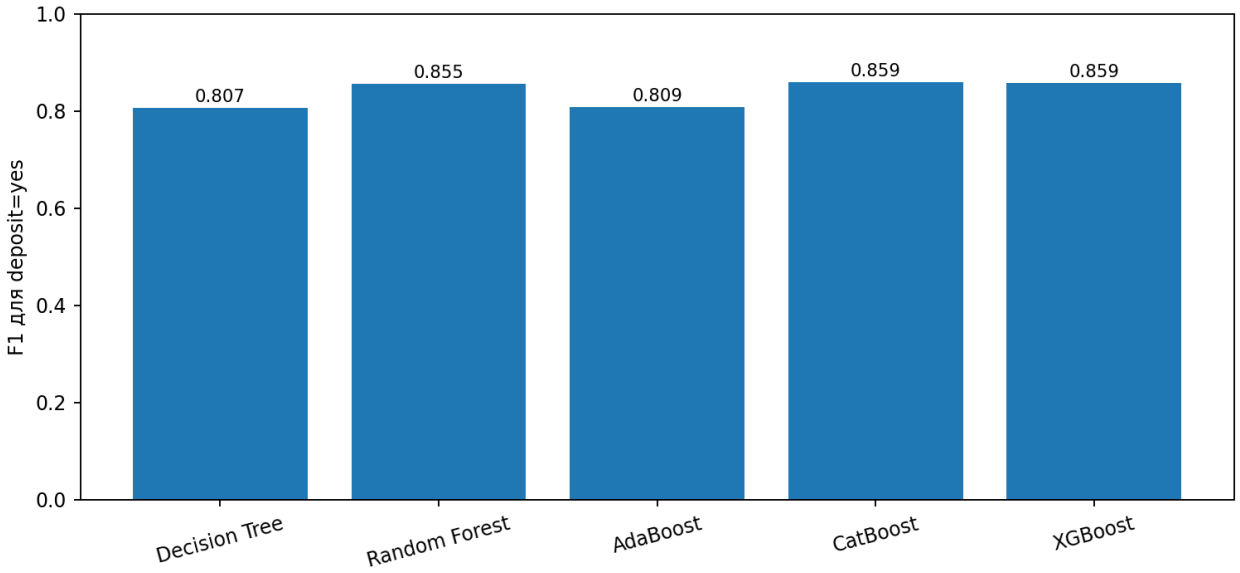
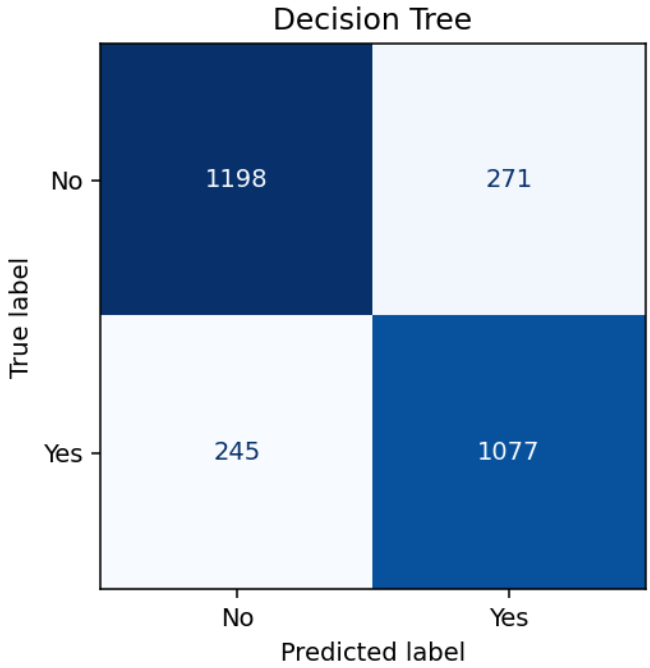
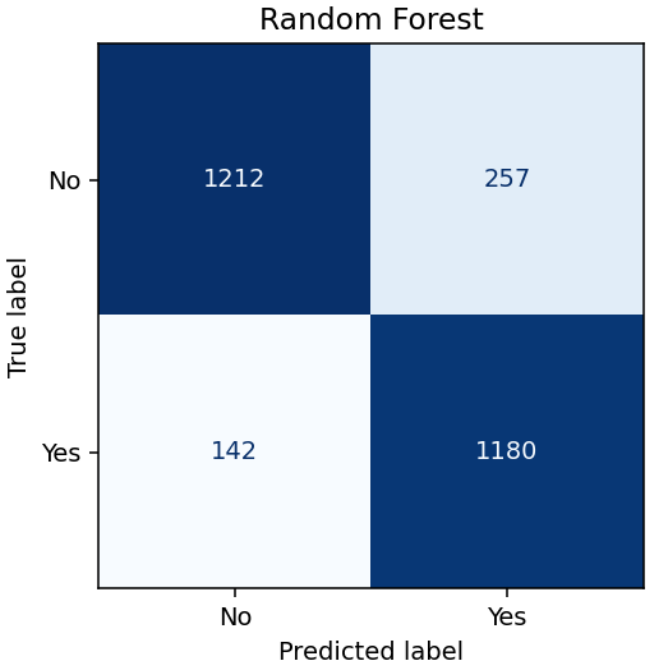


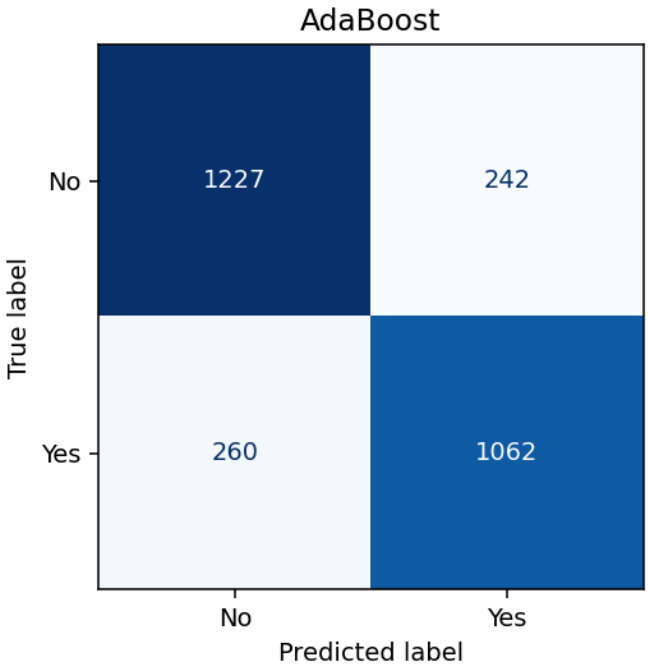
Рисунок — сравнение F1 по алгоритмам.



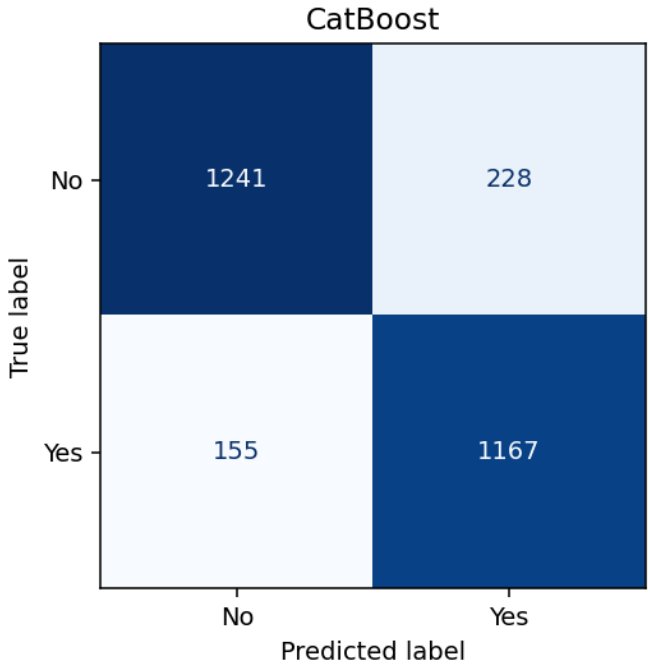
Матрица ошибок модели Decision Tree.



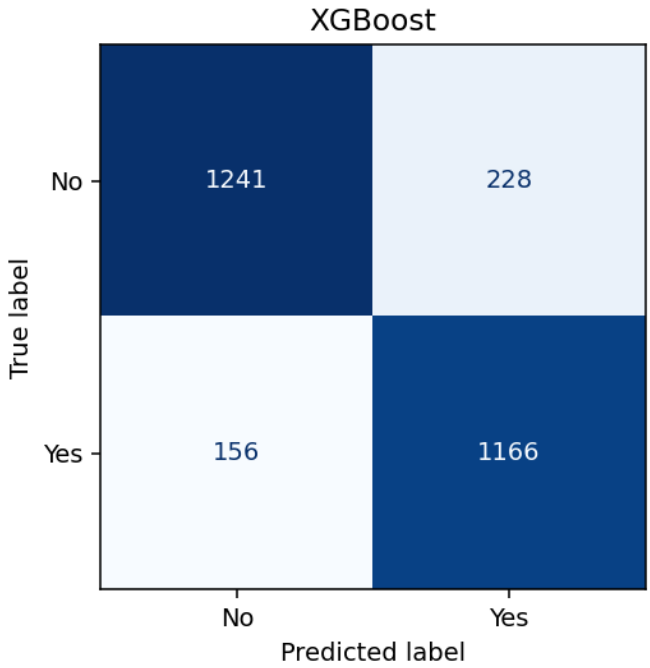
Матрица ошибок модели Random Forest.



Матрица ошибок модели AdaBoost.



Матрица ошибок модели CatBoost.



Матрица ошибок модели XGBoost.

Выводы о проделанной работе

1. Лучшая F1 для положительного класса в данном разбиении: CatBoost — 0.8590; худшая из сравниваемых — Decision Tree (0.8067).
2. Ансамблевые методы могут снижать дисперсию отдельных деревьев, но требуют большего времени настройки; сравнение проведено в одинаковых условиях предобработки и разделения данных.

3. При финансовых решениях важно оценивать не только F1, но и стоимость ложноположительных и ложноотрицательных прогнозов, устойчивость к временным изменениям и доступность признаков до совершения звонка.
4. Итоговые показатели относятся к отложенной выборке и фиксированным гиперпараметрам, поэтому не означают гарантированного качества на новых банковских кампаниях.

Приложение — исходный код

Файл ИАД_5.py: полный исполняемый код размещён также в папке src/.

```
"""ИАД, лабораторная №5, вариант 11, Яшков П. А.: Bank Marketing."""
from pathlib import Path
import json,time
import numpy as np,pandas as pd
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.metrics import
accuracy_score,precision_score,recall_score,f1_score,confusion_matrix,ConfusionMatrixDisplay
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier,AdaBoostClassifier
from xgboost import XGBClassifier
from catboost import CatBoostClassifier
ROOT=Path(__file__).resolve().parent;OUT=ROOT/'results';OUT.mkdir(exist_ok=True)
df=pd.read_csv(ROOT/'data/bank.csv')
assert 'deposit' in df.columns
X=df.drop(columns=['deposit']);y=df.deposit.astype(str).str.lower().eq('yes').astype(int)
cat=X.select_dtypes(include='object').columns.tolist();numeric=X.select_dtypes(exclude='object').columns.tolist
()
Xt,Xs,yt,ys=train_test_split(X,y,test_size=.25,random_state=42,stratify=y)
pre=ColumnTransformer([
 ('n',Pipeline([('fill',SimpleImputer(strategy='median'))]),numeric),

('c',Pipeline([('fill',SimpleImputer(strategy='most_frequent')),('onehot',OneHotEncoder(handle_unknown='ignore'
,sparse_output=False))]),cat)
],sparse_threshold=0)
train=pre.fit_transform(Xt);test=pre.transform(Xs)
models={
 'Decision Tree':DecisionTreeClassifier(max_depth=8,min_samples_leaf=8,random_state=42),
 'Random Forest':RandomForestClassifier(n_estimators=160,min_samples_leaf=2,random_state=42,n_jobs=2),
 'AdaBoost':AdaBoostClassifier(n_estimators=150,learning_rate=.65,random_state=42),

'CatBoost':CatBoostClassifier(iterations=210,depth=5,learning_rate=.075,random_seed=42,verbose=False,thread_cou
nt=2),

'XGBoost':XGBClassifier(n_estimators=210,max_depth=5,learning_rate=.075,subsample=.9,colsample_bytree=.9,random
_state=42,n_jobs=2,eval_metric='logloss')}
metrics={}
for name,model in models.items():
 t=time.perf_counter();model.fit(train,yt);duration=time.perf_counter()-t
 p=model.predict(test); cm=confusion_matrix(ys,p,labels=[0,1]);

metrics[name]={"accuracy":float(accuracy_score(ys,p)),"precision_yes":float(precision_score(ys,p,zero_division=
0)),"recall_yes":float(recall_score(ys,p,zero_division=0)),"f1_yes":float(f1_score(ys,p,zero_division=0)),"conf
usion_matrix":cm.tolist(),"training_seconds":round(duration,3)}

fig,ax=plt.subplots(figsize=(5.1,4));ConfusionMatrixDisplay(confusion_matrix=cm,display_labels=['No','Yes']).pl
ot(ax=ax,cmap='Blues',colorbar=False);ax.set_title(name);fig.tight_layout();fig.savefig(OUT/f'confusion_{name.l
ower().replace(" ","_")}.png',dpi=140);plt.close(fig)
 print(f'{name}: F1(yes)={metrics[name]["f1_yes"]:.4f}, Precision={metrics[name]["precision_yes"]:.4f},
Recall={metrics[name]["recall_yes"]:.4f}')
result={'variant':11,'student':'Яшков П. А.','dataset':'Course Drive Bank Marketing (bank.csv,
deposit)','rows':len(df),'train_rows':len(Xt),'test_rows':len(Xs),'features_encoded':int(train.shape[1]),'targe
t':'deposit=yes','models':metrics}
(OUT/'metrics.json').write_text(json.dumps(result,ensure_ascii=False,indent=2),encoding='utf-8')
fig,ax=plt.subplots(figsize=(9,4.3));names=list(metrics);vals=[metrics[n]['f1_yes'] for n in
names];b=ax.bar(names,vals);ax.set_ylabel('F1 для
deposit=yes');ax.set_ylim(0,1.0);ax.tick_params(axis='x',labelrotation=15)
for rect,val in
zip(b,vals):ax.text(rect.get_x()+rect.get_width()/2,val+.012,f'{val:.3f}',ha='center',fontsize=9)
fig.tight_layout();fig.savefig(OUT/'model_comparison.png',dpi=170);plt.close(fig)
```